AI-Driven Fusion Testing Simulator (AFTS)

Open Architecture Specification

Version 1.0.0 - Public Architecture Edition

An open, implementation-neutral architecture for site-aware AI-assisted multiphysics simulation

Contents

- Open Architecture Specification - Version 1.0.0
- Abstract
- 1. Scope
- 2. Design philosophy
- 3. High-level goals
- 4. Layered architecture
- 5. Canonical data flow
- 6. Site and rotating-frame requirements
- 7. Physics engine contract
- 8. Stability and diagnostic model
- 9. AI requirements
- 10. Uncertainty quantification
- 11. Verification and validation
- 12. Reproducibility package
- 13. Hardware-in-the-loop
- 14. Interfaces and universal design
- 15. Capability profiles
- 16. Limitations and research gaps
- 17. Lucy implementation note
- 18. The AFTS Covenant

AI-Driven Fusion Testing Simulator (AFTS)

Open Architecture Specification - Version 1.0.0

Status: Public Architecture Edition

Scope: Open, implementation-neutral concept and system architecture

Canonical principle: Physics is authoritative; AI is assistive; every result is reproducible and site-aware when required by the modeled sensitivity.

Abstract

AFTS defines a modular architecture for high-fidelity virtual testing of fusion and hybrid acoustic-plasma systems under realistic laboratory conditions, including non-inertial local-frame effects. It combines site and frame services, replaceable physics engines, explicit multiphysics coupling, AI-assisted acceleration and control, uncertainty quantification, immutable provenance, cross-site comparison, and optional hardware-in-the-loop operation.

The specification is not a software implementation and does not prescribe a user interface, programming language, numerical method, deployment environment, AI provider, or product platform. Lucy Fusion Testing Studio is a planned implementation of the architecture, not a prerequisite.

1. Scope

AFTS applies to digital-twin, simulation-campaign, controller-development, educational, and HIL environments that model one or more of the following:

- plasma or magnetohydrodynamic behavior;
- electromagnetic fields and plasma response;
- compressible fluids and acoustic fields;
- acoustic radiation forces and streaming;
- particles, levitated targets, or diagnostic masses;
- rotating-frame and site-dependent effects;
- AI surrogates, anomaly detection, control, UQ, or experiment design.

An implementation may support only a subset, but must identify its supported profile and limitations.

2. Design philosophy

AFTS follows physics before AI, scientific integrity, reproducibility by default, site awareness, open architecture, modular composition, explainable intelligence, verification before

optimization, human authority, long-term preservation, universal accessibility, and continuous evolution.

3. High-level goals

1. 1. Simulate coupled plasma, electromagnetic, acoustic, fluid, and particle behavior at fidelity appropriate to the scientific question.
2. 2. Make local reference-frame inputs explicit, including WGS84 position, local ENU frame, Earth rotation, effective gravity, vertical deflection, and optional tidal or magnetic corrections.
3. 3. Use AI to accelerate or assist, never to silently replace scientific authority.
4. 4. support digital twins and safely bounded HIL operation.
5. 5. Produce reproducible cross-site comparisons with quantified uncertainty.
6. 6. Allow independent implementations to exchange experiment definitions, model identities, results, and provenance.

4. Layered architecture

Layer 0 - Site and Frame Services

Responsibilities:

- ingest latitude, longitude, ellipsoidal height, datum, epoch, and coordinate uncertainty;
- transform global coordinates to a declared local frame, normally East-North-Up;
- compute the local representation of Earth's angular velocity;
- compute or import effective gravity, including declared centrifugal treatment;
- expose optional gravity anomaly, vertical deflection, magnetic declination, tide, and environmental context;
- supply a traceable time basis for phase-coherent coupling and HIL synchronization.

Frame outputs are versioned simulation inputs. Engines must not infer or silently substitute site values.

Layer 1 - Core Physics Engines

AFTS permits independently replaceable engines, including:

- extended MHD, two-fluid, kinetic, or gyrokinetic plasma solvers;
- compressible Navier-Stokes and acoustic solvers;
- acoustic radiation force and streaming modules;
- electromagnetic field solvers;
- discrete particle, target, or diagnostic-mass trackers;
- thermal, structural, materials, or radiation modules where required.

Every engine declares equations, approximations, units, coordinate frame, discretization, stability constraints, boundary conditions, tolerances, and validation status.

Layer 2 - Coupling and Multiphysics Orchestrator

The orchestrator owns coupling order, data exchange, time synchronization, iteration policy, convergence, checkpointing, and failure handling. It may use operator splitting, partitioned iteration, co-simulation, or tightly coupled methods.

Required coupling records include exchanged quantities, interpolation and projection operators, coupling cadence, lag, residuals, convergence thresholds, conservation diagnostics, and rollback behavior.

Examples include acoustic body force entering fluid or plasma momentum, streaming velocity entering Coriolis or advective terms, field response affecting charged-particle motion, and thermal or structural changes altering geometry or boundary conditions.

Layer 3 - AI and Machine-Learning Fabric

Permitted roles include:

- surrogate acceleration using neural operators, graph models, reduced-order models, or hybrid physics-ML methods;
- control recommendation or bounded closed-loop control;
- anomaly, drift, and numerical-instability detection;
- uncertainty estimation and calibration;
- active learning and automated experiment design;
- explanation, report generation, and parameter sensitivity analysis.

Every AI model declares its training data lineage, target quantities, applicability domain, validation metrics, uncertainty behavior, failure modes, model hash, and fallback path. A surrogate must not be presented as equivalent to an authoritative solver outside demonstrated validation bounds.

Layer 4 - Data and Provenance

Each run produces an immutable or tamper-evident record containing:

- experiment, site, geometry, material, plasma, acoustic, and control inputs;
- units and coordinate frames;
- solver and model manifests;
- software versions, source revisions, compiler/runtime information, and hardware context;
- numerical methods, tolerances, mesh identity, random seeds, and checkpoint lineage;
- validation references, warnings, uncertainty, failures, and result hashes.

A cross-site service may replay an experiment under a new site manifest while preserving all other declared parameters and reporting the induced delta.

Layer 5 - Interfaces and Deployment

AFTS may be implemented with any suitable language or framework. Common choices may include Python, C++, Julia, Rust, Fortran, gRPC, ZeroMQ, MPI, shared memory, containers, notebooks, desktop applications, web interfaces, supercomputing workflows, PLCs, or FPGAs.

Deployment technology is not normative. Scientific contracts and declared behavior are.

5. Canonical data flow

7. 1. A human or automated designer creates an experiment manifest containing geometry, operating parameters, transducer or coil configuration, requested models, site, and validation objectives.
8. 2. Site and Frame Services resolve coordinates, frames, gravity, rotation, timing, and associated uncertainty.
9. 3. The orchestrator initializes engines and validates units, frames, meshes, coupling contracts, and capability compatibility.
10. 4. Engines advance the coupled state. Approved surrogates may replace declared substeps inside their validity domain.
11. 5. Diagnostics, modal energies, residuals, conservation measures, stability reports, uncertainty, and anomalies are streamed to observers and controllers.
12. 6. Bounded control recommendations are applied only through declared authority and safety rules.
13. 7. Results and provenance are finalized, hashed, exported, and optionally replayed at alternate sites.

6. Site and rotating-frame requirements

An implementation claiming Site-Aware capability must:

- declare datum, coordinate epoch, latitude convention, longitude convention, and height convention;
- declare the transformation to the local frame;
- expose the local Earth-rotation vector and its units;
- state whether gravity includes centrifugal effects and avoid double counting;
- state the equations in which Coriolis, centrifugal, and Euler terms are included;
- propagate coordinate and site-model uncertainty when material to the output;
- retain a zero-rotation or inertial-limit verification case.

For a local ENU frame at geodetic latitude phi, a common representation is omega_ENU = (0, omega_E cos(phi), omega_E sin(phi)); implementations may use another convention if fully declared and tested.

7. Physics engine contract

Each engine provides conceptual operations equivalent to:

- describe capabilities and governing equations;
- validate configuration;
- initialize state;
- advance by an accepted interval;
- exchange boundary or coupling data;
- compute diagnostics and uncertainty contributions;
- checkpoint and restore;
- finalize and emit provenance.

The exact API is implementation-specific. The behavior and records are portable.

8. Stability and diagnostic model

AFTS does not require one universal scalar stability value. Implementations should produce a structured Stability Report containing:

- plasma stability indicators and dominant modes;
- acoustic-mode stability and mode coupling;
- flow and streaming shear;
- frame-effect contribution or sensitivity;
- controller stability and saturation state;
- numerical stability, residuals, and convergence;
- uncertainty and confidence interval;
- validation state and applicability warnings.

An implementation may publish an aggregate score S, but must retain the component values, normalization, weights, units, and derivation.

9. AI requirements

AI-assisted implementations must preserve a non-AI authoritative path for validation or declare why one is unavailable. Physics-informed constraints are encouraged but do not by themselves establish correctness.

AI control must include operating limits, action rate limits, state observability requirements, safe fallback, uncertainty thresholds, out-of-distribution detection, and complete command provenance.

Continual learning must never silently modify a validated production model. Updated models receive new identities and require revalidation.

10. Uncertainty quantification

UQ should distinguish:

- input uncertainty;
- site and frame uncertainty;
- discretization and numerical error;
- coupling error;
- model-form uncertainty;
- surrogate error;
- measurement and calibration uncertainty;
- stochastic variation.

Results should state whether intervals are frequentist, Bayesian, ensemble-derived, heuristic, or otherwise constructed. Confidence must not be represented by an unexplained percentage.

11. Verification and validation

The verification hierarchy includes:

14. 1. unit and dimensional tests;
15. 2. coordinate and frame transformation tests;
16. 3. analytic limits, including zero-rotation behavior;
17. 4. manufactured solutions with known inertial-force terms;
18. 5. single-physics benchmark tests;
19. 6. coupling and conservation tests;
20. 7. cross-code comparisons;
21. 8. published experimental benchmark replication;
22. 9. laboratory comparison;

10. HIL closed-loop validation.

Round-trip and cross-site tests must identify which state variables may be transformed directly and which require a new solution because the governing environment changed.

12. Reproducibility package

A publishable run package should contain:

- experiment and site manifests;
- geometry and mesh identifiers or archived assets;
- solver and AI model manifests;
- complete configuration with units;
- software and environment record;
- result data and diagnostic summary;

- validation evidence;
- provenance chain and checksums;
- human-readable report;
- license and access restrictions for included data.

13. Hardware-in-the-loop

HIL is optional and high consequence. An AFTS HIL implementation must provide:

- explicit simulation-only, advisory, supervised, and active-control modes;
- authenticated and authorized operators;
- independent physical interlocks and emergency stop;
- bounded commands, rate limits, watchdogs, and timeout-to-safe-state;
- separation between AI recommendation and actuator execution;
- deterministic command validation;
- synchronized telemetry and complete audit records;
- rehearsed loss-of-communications and model-failure behavior.

No simulator or AI layer may be treated as the sole safety mechanism.

14. Interfaces and universal design

AFTS defines information and behavior, not appearance. An implementation may use a command line, notebook, conventional laboratory dashboard, desktop studio, cinematic three-dimensional command environment, VR system, educational interface, or another design.

Every interface must distinguish authoritative data, estimated data, surrogate output, uncertainty, warnings, and presentation-only effects. Visual drama must not alter scientific truth.

15. Capability profiles

- **AFTS Core:** site declaration, one physics engine, reproducible run package, diagnostics, and provenance.
- **AFTS Coupled:** two or more coupled engines, explicit exchange contracts, coupling-error diagnostics, and structured stability reporting.
- **AFTS AI-Assisted:** registered AI model, applicability domain, uncertainty, validation, and deterministic fallback or explicit limitation.
- **AFTS HIL:** authenticated operators, independent safety controls, bounded commands, synchronized telemetry, and auditability.
- **AFTS Reference:** complete conformance evidence, benchmark publication, canonical exchange formats, and reproducible cross-site demonstration.

16. Limitations and research gaps

AFTS is an architecture, not evidence that any particular effect is significant. The relevance of Earth rotation, gravity variation, acoustic forces, or other corrections depends on geometry, time scale, velocity, sensitivity, and measurement capability.

High-fidelity solvers remain approximations. AI models may fail outside their validation domain. Cross-site comparison can reveal predicted sensitivity but does not replace experimental confirmation. Real laboratory systems require independent engineering safety analysis and regulatory compliance.

17. Lucy implementation note

Lucy Fusion Testing Studio may implement AFTS as a native Studio with natural-language mission planning, knowledge integration, governed orchestration, approvals, audit, cinematic visualization, and cross-application workflows. These are implementation features. Lucy and EMMA are not required by AFTS.

18. The AFTS Covenant

The AI-Driven Fusion Testing Simulator is published as an open architectural concept for the advancement of scientific research and engineering.

Its purpose is to encourage reproducible experimentation, transparent methodology, honest uncertainty, and interdisciplinary collaboration.

No single architecture can answer every scientific question. AFTS provides a framework upon which researchers, educators, laboratories, industry, and independent builders may build, evaluate, improve, or challenge new ideas.

The architecture belongs to no single software platform. It is intended to remain implementation-neutral so future generations may adapt it to technologies that do not yet exist.

Knowledge grows through collaboration. Discovery grows through experimentation. Humanity advances when both are shared openly.